МЕТОДИЧНІ ВКАЗІВКИ ДО ВИКОНАННЯ ЛАБОРАТОРНИХ РОБІТ З ДИСЦИПЛІНИ «ЛЮДИНО-МАШИННА ВЗАЄМОДІЯ»

ЛАБОРАТОРНА РОБОТА №1

Розробка програм для реалізації діалогу типу запитання-відповідь

Мета роботи - вивчити особливості розробки структури діалогу типу "запитання-відповідь"

Загальні відомості

Структура діалогу типу запитання-відповідь(Q&A) основана на аналогії з звичайним інтерв'ю. Цей тип діалогу відноситься до діалогу, який керується комп'ютером. В кожній точці діалогу система виводить в якості підказки одне запитання, на яке користувач дає одну відповідь. В залежності від отриманої відповіді система може вирішити, яке наступне запитання задавати. Якщо відповідь невірна система виводить повідомлення про помилку і знову виводить підказку. Цей процес повторюється до тих пір, поки не буде отримано прийнятну відповідь. Структура Q&A не гарантує мінімального об'єму введення, який оцінюється в кількості натискань клавіш. Однак, при відповідному підборі скорочень, надлишковість може бути зменшена. Для прискорення введення системи з структурою Q&A повинні мати механізм випереджального введення відповідей. Хоча структура Q&A з появою потужних засобів відображення інформації дещо застаріла, вона має ряд переваг. Це такі:

- структура Q&A є проміжною структурою, яка може задовольнити вимоги різних користувачів і типів даних;
- вона достатньо гнучка і в достатній мірі забезпечує підтримку користувача;

Ця структура застосовується найбільш часто, коли на кожне запитання передбачена велика кількість відповідей, кожна з яких впливає на зміст наступного запитання. Тому структуру Q&A знайшла широке використання в експертних системах.

Контрольні запитання

1. Критерії оцінки якості діалогу?
2. Які аналоги структури діалогу типу "запитання-відповідь"?
3. Коли найбільш раціонально застосовувати діалог типу Q&A?
4. Дайте визначення діалогу типу Q&A?
5. Хто управляє діалогом в структурі Q&A?
6. Коли використовують випереджувальне введення?
7. Що таке режим ехо-друку?
8. Які види підтримки користувача застосовуються в діалозі типу Q&A.

Завдання

1. Для навчання проектуванню діалогу типу запитання-відповідь перед видачею варіантів завдань рекомендується організувати «письменницький семінар» (writers workshops). Розбити групу на три підгрупи. Перша подає прототип користувацького інтерфейсу з діалогом типу запитання-відповідь, розроблений на папері. Потім друга група розповідає, що їм подобається в цьому прототипі. І, нарешті, третя група пропонує конструктивну критику (тривалість семінару близько 30 хв.). Після цього видати завдання для програмної реалізації (дивись пункти 2-6).
2. Розробити алгоритм і програму зміни атрибутів поля, використовуючи структуру діалогу типу Q&A (варіанти завдань у викладача).
3. Розробити алгоритм і програму виконання типових операцій режиму командного рядка Windows (cmd.exe) з використанням діалогу Q&A (копіювання файлу, зміна імені файлу та ін. - варіанти завдань у викладача).
4. Розробити алгоритм і програму:

- заповнення анкети при прийомі на роботу;
- знімання копії паспорту;
- копії свідоцтва про народження,

та інших документів по завданню викладача.

5. Передбачити в програмі необхідну підтримку користувача, механізми випереджувального введення відповідей, скорочення, виділення кольором, звуком та інше [2-4]. При оцінюванні програми буде враховуватись як функціональність так і юзабельність.
6. Описати діалог за допомогою мережі переходів.
7. Продемонструвати роботу програми та підготувати звіт.

Зауваження. Допускається розробка та реалізація самостійного завдання студентом.

Рекомендоване мова програмування C++, середовище програмування Visual Studio 2008 (2010), тип проекту – Win32 Console Application.

Література

1. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.
2. Інтерфейс "Користувач-комп'ютер": Навчальний посібник / В.П. Майданюк, А.М. Пєтух. - Вінниця: ВДТУ, 1999. - 66 с.
3. Мандел Т. Разработка пользовательского интерфейса / Т. Мандел. - Пер. с англ. – М.: ДМК Пресс, 2001. – 416 с.
4. Коутс Р. Интерфейс "Человек - компьютер" / Р. Коутс, И. Влейминк. - Пер. с англ. - М.: Мир, 1990. – 501 с.
5. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.

ЛАБОРАТОРНА РОБОТА №2

Програмна реалізація структур діалогу командного типу, меню і екранних форм

Мета роботи - вивчення особливостей програмної реалізації структур діалогу командного типу, меню і екранних форм

Загальні відомості

Структура діалогу на основі командної мови відноситься до діалогу, що керується користувачем. Команда включає в себе постановку задачі, можливі пояснення і дані, необхідні для її вирішення. Діалог на базі команд працює в телетайпному режимі. Система нічого не виводить крім постійної підказки, Наприклад, підказка "ім’я пристрою", прийнята для більшості мікрокомп’ютерних ОС. Вона означає готовність системи до роботи. Кожну команду вводять з нового рядка і закінчують натисканням клавіші "ENTER". Наприклад,

C:\ DIR <ENTER>

Відповідальність за правильність команд лягає на користувача. Формат команди в загальному випадку приведений на рис. 1.

Рис. 1. Формат команди

Параметри в списку можна задавати в одному з двох форматів [2]:

- в вигляді позиційних параметрів;
- в вигляді ключових параметрів.

Значення параметра, який задається в позиційній формі, визначається за його місцем в командному рядку. Наприклад, в команді

COPY thisfile newfile

перший параметр задає файл, який повинен копіюватись, а другий параметр, файл куди необхідно копіювати вміст першого. У випадку ключових параметрів кожне значення параметра позначається ідентифікатором, який задає його призначення. Наприклад, в команді

COPY SOURCE=thisfile DESTINATION=newfile

ключові слова source (джерело) і destination (одержувач) визначають призначення кожного параметра. Недоліком позиційних параметрів є те, що помінявши місцями параметри можна отримати протилежний результат, а недоліком ключових параметрів є необхідність запам’ятовування великої кількості ключових слів.

Список параметрів може містити ключі, які уточнюють спосіб інтерпретації команди. Ключі помічаються спеціальним ідентифікатором (наприклад "-", "/").

Структура діалогу на основі мови команд за своїми можливостями найшвидша і найгнучкіша із всіх структур діалогу, але для роботи в цьому режимі необхідна хороша підготовка користувача.

Структура діалогу типу "меню" характеризується тим, що у користувача є список можливих варіантів для введення, із якого необхідно вибрати ті, що потрібно. Популярність меню особливо виросла в зв’язку з появою інтерфейсу, що керується маніпулятором типу "миша" і появою "віконної" техніки [5,6] виведення даних. Існують різні формати представлення меню на екрані. Деякі з них приведені на рис. 2.

Персонал	***Клієнти***	***Меблі***	***Пристрої***	***Різне***	***Кінець***

а)

Персонал
Клієнти
Меблі
Пристрої
Різне
Кінець

б)

1. ***Персонал***
2. ***Клієнти***
3. ***Меблі***
4. ***Пристрої***
5. ***Різне***
6. ***Кінець***

в)

Рис. 2. Приклади різних форматів "меню". а - меню в вигляді рядка даних; б - меню в вигляді блоку даних; в - меню в вигляді списку об'єктів, які вибираються указанням цифрових кодів

Меню в вигляді блоку даних на екрані традиційний формат. Меню в вигляді рядка даних може появлятись вверху або внизу екрана і часто залишається в цій позиції протягом діалогу. У випадку, коли на екрані недостатньо місця, додаткові меню "випадають" або "вискакують" (POP-UP) на екран в поточній позиції екрана або витягуються (PULL-DOWN) з рядка меню. Ці меню зникають після вибору варіанта. Меню у вигляді піктограм - це множина блоків об'єктів вибору, розкиданих по екрану, які часто містять графічне представлення об'єктів роботи. Користувач діалогового меню може вибрати необхідний пункт вводячи текстовий рядок, який ідентифікує цей пункт; указуючи на нього безпосередньо, наприклад за допомогою маніпулятора типу "миша", або продивляючись список в режимі ролика і вибираючи з нього. Основні переваги і недоліки меню такі:

1. Меню використовується для введення значень даних, діапазон яких невеликий і їх можна згрупувати в ієрархію.

2. Меню природна структура при роботі з пристроями указання і вибору (наприклад маніпулятор типу "миша").

3. Функціональні клавіші можна налаштувати так, щоб кожна з них відповідала пункту меню.

4. Просто сформувати повідомлення про помилки, оскільки прийнятні варіанти відповідей відображені на екрані.

5. Меню можуть використовувати користувачі будь-якої кваліфікації.

Структура діалогу типу "екранна форма" характеризується тим, що система отримує від користувача всю інформацію зразу способом заповнення форми. Аналогом екранної форми є звичайна анкета. На практиці форми використовуються там, де облік якої-небудь діяльності вимагає введення достатньо стандартного набору даних. Користувач може редагувати відповіді, пропускати, повертатись назад. Комп'ютерна система може перевіряти кожне введене повідомлення зразу після введення або після заповнення всієї форми. Деякі характеристики цієї структури:

1. Працює швидше, ніж Q&A.
2. В порівнянні з "меню" може маніпулювати більш широким набором вхідних даних.
3. В меншій мірі підходить для роботи в режимі вибору варіантів, оскільки має послідовну, а не деревоподібну організацію.
4. Знаходить застосування в базах даних.
5. Може слугувати користувачам будь-якої кваліфікації.

Кожна з цих структур ґрунтується на аналогії взаємодії між людьми. Ці три структури діалогу можна розглядати як різновидності структури запитання - відповідь. Їх можливо представити таким чином:

- меню : опереджувальна довідка;

- форма : опереджувальне запитання;
- команда : опереджувальне введення.

Не дивлячись на те, що більшість систем мають в своїй основі структури діалогу типу запитання-відповідь, меню або структуру на основі команд, рідко вдається побудувати діалог для всієї системи, використовуючи тільки одну структуру. Для різних частин діалогу необхідні різні структури в залежності від їх конкретних характеристик, тобто більшість діалогів ґрунтується на змішаних структурах.

Контрольні запитання

1. Які користувачі можуть ефективно використовувати діалог командного типу?
2. Які критерії розробки діалогу командного типу?
3. Як пов'язані між собою діалог командного типу і діалог типу запитання-відповідь(Q&A).
4. Що включає в себе команда?
5. Які складові частини інтерфейсу людина-комп'ютер?
6. Критерії розробки "меню"?
7. Які формати використовуються для виводу "меню" на екран?
8. Охарактеризуйте "меню" як різновидність діалогу типу Q&A?
9. Які користувачі найбільш часто використовують діалог типу "меню"?
10. Наведіть основні характеристики структури діалогу типу "екранна форма".
11. В яких випадках використовується структура діалогу типу "екранна форма"?

Завдання.

1. Розробити алгоритм і програму виконання таких команд в режимі діалогу командного типу, типу Q&A (використовуєте модулі з лабораторної роботи №1), меню і екранна форма (анкета)":

1. COMP - порівняння файлів;
2. DISKCOMP - порівняння дискет;
3. MODE - встановлення режимів системних пристроїв ;
4. DATE - встановити дату;
5. DIR - вивести каталог;
6. ERA - видалити файл, або файли з диску;
7. PIP - скопіювати файл;
8. REN - перейменувати файл;
9. STAT - вивести дані про використання дискової пам'яті;
10. TIME - встановити поточну дату;
11. TYPE - вивести вміст файлу на екран.

Варіанти

1. 1,4,5*
2. 2,5,6
3. 3,6,7
4. 4,8,9
5. 5,9,10
6. 6,10,11
7. 7,8,9
8. 8,1,11
9. 9,2,6
10. 10,3,7.

* Перший варіант, студент реалізує команди 1, 4, 5 - COMP - порівняння файлів; DATE - встановити дату; DIR - вивести каталог.

Мнемоніка команд може змінюватись студентами, виходячи з необхідності забезпечення якості діалогу. Для командного режиму параметри вводяться з командного рядка, а якщо користувач не ввів параметри, то для їх введення використовується структура діалогу типу Q&A.

Приклад:

A > TIME 20 40 30 [ENTER] //Встановлює час 20 год. 40 хв. 30 сек.

A > TIME [ENTER]
/год/хв./сек: 20/40/30 [ENTER] //Теж саме
A >

Вибір типу діалогу виконується за допомогою меню у вигляді списку об'єктів, які вибираються введенням цифрових кодів.

2. Описати діалог за допомогою мережі переходів.
3. Виконати розрахунок часу, який необхідний оператору для виконання тої чи іншої дії (команди) для різних типів діалогу з використанням моделі GOMS. Розрахунок виконати згідно з прикладом наведеним в [1] (стр. 68-80, файл hmi_lections.pdf в директорії book]. Зробити висновки.
3. Оформити звіт по лабораторній роботі.

Зміст звіту:

- алгоритм програми;
- опис діалогу за допомогою мережі переходів;
- розрахунок часу згідно моделі GOMS.

- текст програми.

Зауваження. Допускається розробка та реалізація самостійного завдання студентом по узгодженню з викладачем.

Рекомендоване мова програмування C++, середовище програмування Visual Studio 2008 (2010), тип проекту – Win32 Console Application.

Література

1. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.
2. Інтерфейс "Користувач-комп'ютер": Навчальний посібник / В.П. Майданюк, А.М. Пєтух. - Вінниця: ВДТУ, 1999. - 66 с.
3. Мандел Т. Разработка пользовательского интерфейса / Т. Мандел. - Пер. с англ. – М.: ДМК Пресс, 2001. – 416 с.
4. Коутс Р. Интерфейс "Человек - компьютер" / Р. Коутс, И. Влейминк. - Пер. с англ. - М.: Мир, 1990. – 501 с.

ЛАБОРАТОРНА РОБОТА №3

Елементи керування проекта Windows Form Visual Studio

Мета роботи - вивчити особливості розробки графічного інтерфейса користувача з використанням Windows Form.

Загальні відомості

Проект Windows Forms Application дозволяє автоматизувати проектування інтерфейсу користувача за рахунок застосування форм Windows подібно візуальним компонентам Delphi [2].

Бібліотека Windows.Forms надає прості й у той же час потужні механізми для керування графічним інтерфейсом користувача.

Форма являє собою екранний об'єкт, який можна застосовувати для надання інформації користувачу і для обробки введення інформації від користувача. Найпростіший спосіб задати інтерфейс користувача для форми - розмістити елементи керування на її поверхні. З іншого боку, форма - це об'єкт, що задається властивостями, що визначають їхній зовнішній вигляд, методами, що визначають їхню поведінку, і подіями, що визначають їхню взаємодію з користувачем. Форми, як і всі візуальні об'єкти в .NET, є екземплярами класів. Зокрема, форми успадковані від System.Windows.Forms.Form.

При проектуванні додатків графічного інтерфейсу користувача програміст взаємодіє зі спеціальним засобом - дизайнером форм. Дизайнер форм призначений для візуальної розробки користувальницького інтерфейсу програми. До основних елементів дизайнера форм можна прилічити:

- Properties Window (пункт меню View /Properties Window);
- Layout Toolbar (пункт меню View/Toolbars/Layout);
- Toolbox (пункт меню View/Toolbox).

Проектування графічного інтерфейсу здійснюється аналогічно відповідним технологіям Visual Basic, Delphi, тощо.

Припустимо, нам необхідно створити програму графічного інтерфейсу користувача для роботи з даними про книги на книжковій полиці. Необхідно завантажувати дані з обраного файлу, відображати дані про книги у вигляді таблиці, відображати додаткові дані в окремому підвікні, дозволяти редагування даних, сортувати книги за назвою, знаходити книги з певним автором та дозволяти зберігати змінені дані в новому файлі.

У нашому випадку програма графічного інтерфейсу користувача міститиме таблицю, яка відображає дані про книги, кнопки, пов'язані з конкретними функціями, область відображення даних про авторів.

Після створення нового Windows-додатку на екрані з'являється вікно дизайнера форм. У вікні "Properties" можна налаштувати властивості головного вікна, зокрема властивості Text встановити значення "Книжкова полиця". Далі можна проектувати інтерфейс користувача шляхом додавання до форми візуальних компонентів, палітра яких міститься у вікні "Toolbox" (View | Toolbox).

Спочатку до форми доцільно додати панель (Panel) для розміщення кнопок. Панель знаходиться на закладці "Containers" вікна "Toolbox". У вікні властивостей (Properties) вставнолюємо для властивості Dock значення Top. Кнопки (компоненти типу Button) знаходяться на закладці "Common Controls". Напис на кнопці можна змінити, редагуючи властивість text. Між третьою та четвертою кнопками додаємо компонент TextBox - рядок, необхідний для введення прізвища автора, якого ми шукаємо. Цей компонент також знаходиться на закладці "Common Controls".

Для того, щоб кнопки і рядок введення були однакового розміру,були розташовані на однаковій відстані одна від одної, симетрично відносно панелі, слід їх одночасно виділити (наприклад, за допомогою клавіші Shift), а потім застосувати функції підменю Format головного меню: Align | Tops,

Make Same Size | Both, Horizontal Spacing | Make Equal, Center in Form | Horizontally та Center in Form | Vertically.

Наступний крок - додавання компоненту DataGridView. Це - універсальний компонент для відображення та редагування табличних даних. Він знаходиться на закладці "Data". У першій з доданих таблиць (вона міститься в нижній частині форми) відображатимуться дані про авторів, тому її назву з dataGridView1 (властивість Name у таблиці властивостей) доцільно змінити на dataGridViewAuthors. Для цього компоненту слід налаштувати значення властивості Dock (встановити у Bottom).

Тепер до форми можна додати ще один компонент DataGridView для відображення книжок. Його ім'я слід встановити у dataGridViewBooks. У вікні властивостей встановлюємо для властивості Dock значення Fill.

В результаті форма матиме такий вигляд:

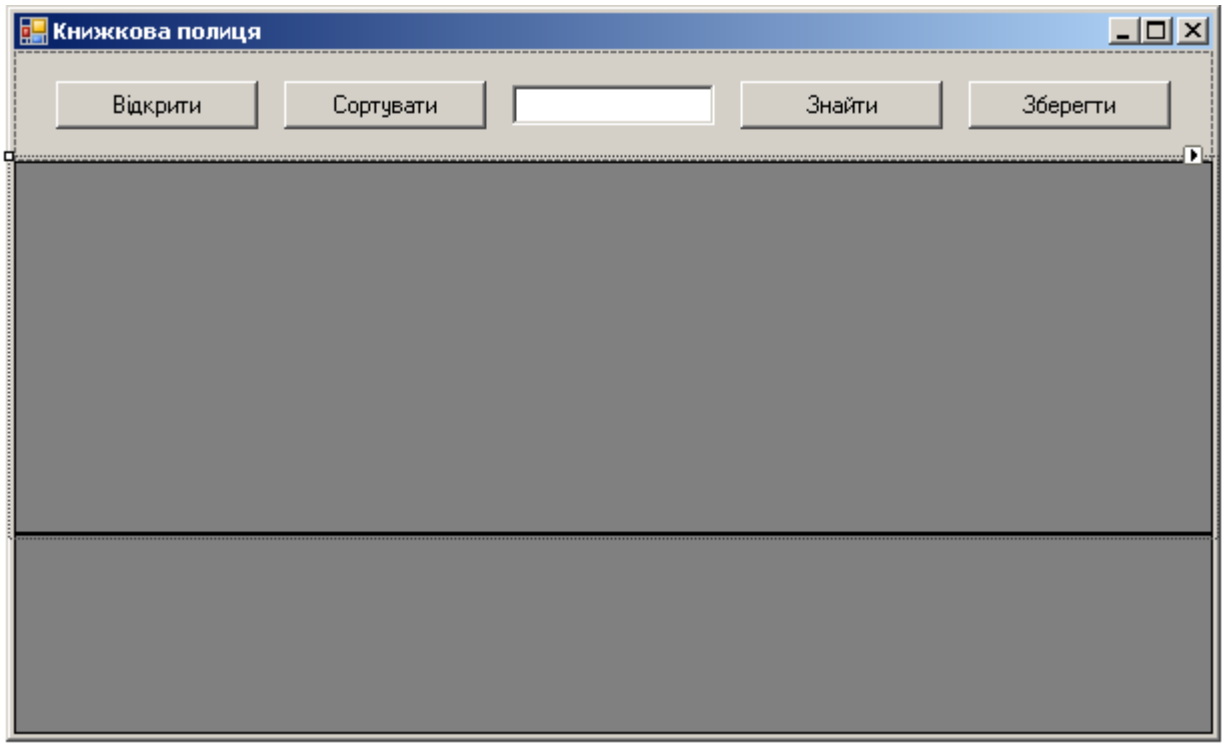

Рисунок 1 - Форма, яка проектується

Для реалізації функці завантаження з файлу та зберігання в іншому файлі до вікна дизайнеру форм слід додати діалогові вікна OpenFileDialog та SaveFileDialog, які знаходяться на закладинці Dialogs палітри. Відповідні діалогові компоненти додаються до сірої панелі в нижній частині вікна

дизайнеру. Для редагування таблиці (збільшення та зменшення кількості рядків) додаємо контекстне меню (компонент типу ContextMenuStrip). Цей компонент знаходиться на закладці "Menus & Toolbars". Він також додається до панелі в нижній частині вікна дизайнеру. Після подвійного натискання лівої кнопки миші майбутнє контекстне меню розкривається на головній формі. До нього послідовно додаємо рядки з наступним змістом: "Додати рядок" та "Видалити рядок". Для того, щоб запобігти появі символів кирилиці серед імен змінних, відповідні імена компонентів слід змінити на addRowToolStripMenuItem та removeRowToolStripMenuItem.

Примітка: для спрощення прикладу ми будемо додавати книги лише з одним автором. Додати можливість зміни кількості авторів можна аналогічно зміні кількості книжок.

Після проектування форми можна перейти до редагування коду. Для переходу на сторінку вихідного коду можна скористатися функцією View Code контекстного меню сторінки дизайнеру. Частина класу, яка призначена для ручного додавання коду зазвичай має такий вигляд:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;

namespace BookshelfApp
{
  public partial class Form1 : Form
  {
    public Form1()
    {
```

```
        InitializeComponent();
      }
    }
  }
```

Тепер необхідно реалізувати оброблювачі подій для кнопок. Для додавання оброблювача події треба обрати необхідну кнопку та у вікні "Properties" натиснути кнопку "Events" (з блискавкою). Після подвійного натискання лівої кнопки миші у рядку "Click" можна редагувати код функції-оброблювача, який з'явився у головному вікні. Наприклад, для першої кнопки він спочатку буде таким:

```
private void button1_Click(object sender, EventArgs e)
{
  if (openFileDialog1.ShowDialog() == DialogResult.OK)
  {


  }
}
```

Всередині тіла функції додаємо код, у якому відображаємо діалогове вікно відкриття файлів. Якщо файл було обрано (результат функції ShowDialog() дорівнює OK), книжки завантажуються з файлу. У випадку невірного формату файлу (функція readFromXML() повернула false) відтворюється початковий стан. Функція MessageBox.Show() з параметрами, які визначають текст та заголовок вікна, виводить відповідне повідомлення. В обох випадках здійснюється повторна ініціалізація таблиці:

```
private void button1_Click(object sender, EventArgs e)
{
  if (openFileDialog1.ShowDialog() == DialogResult.OK)
  {
    if (shelf.readFromXML(openFileDialog1.FileName))
    {
```

```
        dataGridViewBooks.DataSource = null;
      }
      else
      {
        shelf = new XMLShelf();
        Book book = new Book();
        book.Authors.Add(new Author());
        shelf.addBook(book);
        MessageBox.Show("Помилка читання з файлу", "Помилка");
      }
      initGrid();
    }
  }
```

Для того, щоб відомості про авторів відображались у другій таблиці, слід додати обробку події RowEnter першої таблиці. Ця подія виникає, коли активізується рядок з певною книгою. Оброблювач цієї події буде таким:

```
  private void dataGridViewBooks_RowEnter(object sender, DataGridViewCellEventArgs e)
  {
    int index = e.RowIndex;
    dataGridViewAuthors.DataSource = shelf[index].Authors;
    dataGridViewAuthors.Columns["Name"].DisplayIndex = 0;
    dataGridViewAuthors.Columns["Name"].HeaderText = "Ім\'я";
    dataGridViewAuthors.Columns["Surname"].DisplayIndex = 1;
    dataGridViewAuthors.Columns["Surname"].HeaderText = "Прізвище";
    if (shelf[index].Authors.Count == 0)
      shelf[index].Authors.Add(new Author());
  }
```

Для того, щоб здійснювати додавання та видалення рядків, буде застосоване котекстне меню. Для того, щоб воно відображалося після

натискання правої кнопки миші на основній таблиці, необхідно властивості ContextMenuStrip компоненту dataGridViewBooks встановити значення contextMenuStrip1. Далі для окремих позицій контекстного меню (addRowToolStripMenuItem та removeRowToolStripMenuItem) створюємо функції-оброблювачі подій. До цих функцій додаємо дії, пов'язані з додаванням та видаленням рядків. Отримаємо такий код:

```
private void addRowToolStripMenuItem_Click(object sender, EventArgs e)
{
  dataGridViewBooks.DataSource = null;
  Book book = new Book();
  book.Authors.Add(new Author());
  shelf.addBook(book);
  initGrid();
}

private void removeRowToolStripMenuItem_Click(object sender, EventArgs e)
{
  if (shelf.Books.Count > 1)
  {
    dataGridViewBooks.DataSource = null;
    shelf.Books.RemoveAt(shelf.Books.Count - 1);
    initGrid();
  }
}
```

Сортування списку з автоматичним відображенням пов'язане з кнопкою button2:

```
private void button2_Click(object sender, EventArgs e)
{
```

```
    dataGridViewBooks.DataSource = null;
    shelf.sortByName();
    initGrid();
  }
```

Для відображення результатів пошуку книг з певним автором можна створити та додати до проекту нову форму (Функція меню Project | Add Windows Form...). До нової форми (Form2) додаємо компонент TextBox. Для цього компоненту слід налаштувати значення властивості Dock (встановити у Fill), а також ReadOnly та Multiline (встановити значення true).

Припусимо, у класі XMLShelf реалізована функція findByAuthor():

```
  public List<string> findByAuthor(string surname)
  {
    List<string> found = new List<string>();
    foreach (Book book in Books) {
      foreach (Author author in book.Authors) {
        if (author.Surname.Equals(surname))
        {
          found.Add(book + "");
          break;
        }
      }
    }
    return found;
  }
```

У файлі Form2.cs до коду класу Form2 додаємо функцію setContent():

```
  public void setContent(string text)
  {
    textBox1.Text = text;
  }
```

Тепер можна додати оброблювач події button3:

```
private void button3_Click(object sender, EventArgs e)
{
  string surname = textBox1.Text;
  Form2 form2 = new Form2();
  form2.Text = "Автор: " + surname;
  List<string> found = shelf.findByAuthor(surname);
  string text = "";
  foreach (string book in found)
    text += book + Environment.NewLine;
  form2.setContent(text);
  form2.ShowDialog();
}
```

Назва форми Form2 визначається в залежності від того, якого автора ми шукаємо.

Для зберігання даних створюється оброблювач натискання на кнопку button4:

```
private void button4_Click(object sender, EventArgs e)
{
  if (saveFileDialog1.ShowDialog() == DialogResult.OK)
  {
    if (!shelf.writeToXML(saveFileDialog1.FileName))
      MessageBox.Show("Помилка запису в файл", "Помилка");
  }
}
```

Тепер проект BookshelfApp можна запустити на виконання.

Контрольні запитання

1. Концепція програмування, керованого подіями. Поняття події.
2. Для чого потрібні засоби Windows Forms.
3. Як використовують компонент DataGridView.

4. Що таке CheckBox та RadioButton?
5. Де використовується vScrollBar?
6. Для чого використовується компонент TextBox.
7. Назвіть всі відомі Вам елементи керування Windows Forms та їх призначення.

Завдання

1. Створити проект Windows Form і почергово розміщуючи елементи керування Windows Form на головній формі з використанням ToolBox ознайомитись з їх властивостями, використовуючи вкладку Propeties та довідкову систему Visual Studio (викликається по натисканні клавіші F1).
2. Створити оброблювач події, який би змінював певну властивість елемента керування Windows Form під час виконання програми. Наприклад, розмістіть на головній формі елемент Button, створіть обробник при клацанні мишею на даному елементі і попробуйте змінити колір головної форми.
3. Розробити та реалізувати з використанням проекту Windows Form шаблон інтерфейсу користувача робочого місця для:

Варіанти завдань

1. Інтернет-магазин. Мають бути реалізовані сценарії: покупка товару, пошук товару, додавання нового товару в базу даних магазина, перегляд і обробка замовлень покупців, реєстрація нового покупця.

2. Книжковий каталог. Мають бути реалізовані сценарії: додавання нової книги, пошук книги по декількох полях, бронювання книги, списання старих книг, реєстрація користувачів каталогу.

3. Адресна книга. Мають бути реалізовані сценарії: додавання нового абонента, додавання категорій абонентів, пошук абонентів по

декількох полях, додавання адміністраторів каталогу (користувачів які мають право редагувати дані адресної книги), редагування даних абонента.

4. Розклад занять. Мають бути реалізовані сценарії: додавання нової групи, додавання занять (з вказівкою назви предмету, часу, аудиторії, групи, тижня, викладача, типа заняття), перегляд списку занять на вибрану дату, додавання списку викладачів, пошук занять по декількох полях (предмету, викладачу, групі, часу, типові заняття).

5. База студентів. Мають бути реалізовані сценарії: додавання нової групи, додавання нового студента, пошук студента по різних полях, додавання інформації про оцінки по різних предметах, відрахування студента.

6. Прайс-лист фірми. Мають бути реалізовані сценарії: додавання нової категорії товарів, додавання нового товару, пошук товару по різних полях, додавання адміністратора прайс-листа (користувачів які мають право редагувати прайс-лист), переміщення товару з однієї категорії в іншу.

7. База складу фірми. Мають бути реалізовані сценарії: додавання нового товару на склад, списання товару, видача товару, пошук товару по різних полях, зміна місцерозташування товару на складі.

8. Аптечна база. Мають бути реалізовані сценарії: прийом замовлення від клієнта на виготовлення розчину, продаж ліків, списання прострочених ліків, додавання нових ліків в базу даних, пошук заказів по різних полях.

4. Прототип інтерфейсу повинен відповідати основним принципам проектування інтерфейса, забезпечувати необхідну підтримку користувача, при необхідності механізми випереджувального введення відповідей, скорочення, виділення кольором, звуком та

інше [1-4]. При оцінюванні програми буде враховуватись як функціональність так і юзабельність.

5. Перед реалізацією розглянути декілька варіантів розміщення форм на екрані. Порівняти їх з використанням моделі моделі GOMS. Розрахунок виконати згідно з прикладом наведеним в [1] (стр. 68-80, файл hmi_lections.pdf в директорії book). Розрахунок часу на переміщення курсору з використанням маніпулятора типа «миша» виконати згідно законів Фітса і Хіка (стр. 93-95, файл hmi_lections.pdf в директорії book).
6. Продемонструвати роботу програми та підготувати звіт.

Рекомендоване мова програмування C# (C++), середовище програмування Visual Studio 2008 (2010), тип проекту – Windows Form.

Література

1. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.
2. Інтерфейс "Користувач-комп'ютер": Навчальний посібник / В.П. Майданюк, А.М. Пєтух. - Вінниця: ВДТУ, 1999. - 66 с.
3. Мандел Т. Разработка пользовательского интерфейса / Т. Мандел. - Пер. с англ. – М.: ДМК Пресс, 2001. – 416 с.
4. Коутс Р. Интерфейс "Человек - компьютер" / Р. Коутс, И. Влейминк. - Пер. с англ. - М.: Мир, 1990. – 501 с.
5. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.
6. Буч Грейди, Рамбо Джеймс, Джекобсон Айвар. Язык UML. Руководство пользователя. М.: ДМК, 2000
7. Троелсен Э. C# и платформа .NET. Библиотека программиста. / Пер. с англ. - СПб.: Питер, 2002. - 800 с.

ЛАБОРАТОРНА РОБОТА №4

1 Мета роботи: Створення прототипу веб-інтерфейсу з використанням проекту ASP.NET Web Application Visual Studio.

2 **Методичні вказівки**

ASP.NET файл є текстовим файлом і може містити коди HTML, XML і мов сценаріїв [6]. Коди останніх виконуються на веб сервері. Файл ASP.NET має спеціальне розширення ".aspx".

Порядок роботи ASP.NET виглядає таким чином:

- коли веб-браузер запрошує файл ASP.NET, веб-сервер IIS перенаправляє запит модулю ASP.NET на сервері;
- модуль ASP.NET читає файл порядково і виконує, коди сценаріїв, що містяться у файлі;
- веб-браузеру повертається назад файл ASP.NET, але вже у вигляді звичайного документа HTML.

Будь-яка сторінка ASP.NET представлена класом, похідним від класу System.Web.UI, який визначає властивості, методи і події, загальні для всіх сторінок, призначених для обробки середовищем ASP.NET.

Найбільш важливі властивості цього об'єкту приведені в таблиці нижче:

Властивість	**Опис**
Application	Повертає об'єкт HttpApplicationState
Cache	Повертає об'єкт Cache, в якому зберігаються дані додатку, зокрема і самої сторінки
IsPostBack	Повертає значення, що визначає, чи була сторінка завантажена клієнтом вперше, або завантажена повторно у відповідь на запит клієнта
Request	Повертає об'єкт HttpRequest, що використовується для здобуття інформації про вхідний запит HTTP
Response	Повертає об'єкт HttpResponse, що використовується для формування відповіді сервера клієнтові
Server	Повертає об'єкт Http Server Utility
Session	Повертає об'єкт System.Web.SessionState.HttpSessionState, за допомогою якого виводиться інформація про поточний сеанс HTTP

Така побудова проекту дозволяє зберігати окремо код подання для генерації HTML коду (у файлі *.aspx) від програмної логіки (у файлі *.aspx.cs), що у багатьох випадках істотно спрощує розробку складних веб-додатків.

Порядок створення додатка на ASP.NET

1. Створення нового проекту в середовищі Microsoft Visual Studio з використанням шаблону ASP.NET Web Application.

2. Кожен з веб-додатків для IIS повинно розміщуватися в своєму віртуальному каталозі, якому відповідає фізичний каталог на диску. Для створення віртуального каталога необхідно (необов'язково, оскільки Visual Studio містить власну емуляцію сервера):

- відкрити оснащення Microsoft Internet Information Services (Пуск> Панель управления>Администрирование > Internet Information Services);
- розкрити гілку "Веб-вузли" і, перейшовши в "Веб-вузол за умовчанням", створити новий віртуальний каталог;
- у властивостях створеного віртуального каталога вибрати закладку "Документи" і додати в список документів, використовуваних за умовчанням, документ Default.aspx.

3. Після компіляції проекту (опція меню "Build" або <Shift+F6>) і його виконання (<Ctrl+F5>) можна буде побачити в браузері результат виконання сценарію. Після завершення створення проекту, він міститиме файли Default.aspx, Default.aspx.cs і Default.designer.cs.

Серверні елементи управління ASP.NET

Важливою особливістю ASP.NET є використання серверних елементом управління на веб-сторінці (елементи WebForm), які є фактично тегами, зрозумілими веб-серверу. Ці елементи визначені в просторі імен System.Web.UI.WebControls.

Прийнято виділяти три типи серверних елементів управління:

- серверні елементи управління HTML - звичайні теги HTML;
- елементи управління веб-сервера - нові теги ASP.NET;
- серверні елементи управління для перевірки даних (валідації) - застосовуються для валідації вхідних даних від клієнтського застосування (зазвичай веб-браузера).

Переваги від використання таких елементів при розробці веб-додатків:

- скорочується кількість коду, написаного вручну (що особливо помітно для складних елементів документа). Елемент просто "перетягується" з панелі інструментів, після чого виконується налаштування його параметрів в спеціальному вікні. При цьому всі зміни автоматично заносяться безпосередньо в *.aspx файл;
- з програмної точки зору кожному з цих елементів управління відповідає певний клас в бібліотеці базових класів .NET, що дозволяє писати для них такий же код як і для будь-яких інших класів;
- для будь-якого елемента управління WebForm визначений набір подій, які обробляються на веб-сервері;
- для будь-якого елементу управління WebForm надається можливість для перевірки введення даних користувачем.

За умовчанням серверні елементи управління HTML в ASP.NET файлах розглядаються як текст. Для їх програмування потрібне додавання атрибуту runat="server" у відповідний елемент HTML. Крім того, всі серверні елементи управління HTML мають бути розміщені усередині області дії тега <form>, що також має атрибут runat="server".

Подібно до серверних елементів управління HTML елементи управління веб-сервера також створюються на веб-сервері і передбачають додавання атрибута runat="server". Проте вони можуть і не відповідати

конкретним елементам HTML і представляти складніші елементи. Загальний синтаксис для опису таких елементів:

<asp:тип_елемента id="ідентифікатор" runat="server"/>

Серверні елементи валідації застосовуються для перевірки даних, що вводяться користувачем і мають аналогічний синтаксис.

Серверні елементи управління для перевірки даних (валідації)

Елементи управління даного типа застосовуються для перевірки даних, що вводяться. Мають наступний синтаксис:

<asp:тип_елемента id="ідентіфікатор" runat="server"/>

Найбільш важливі елементи наводяться в наступній таблиці.

Елемент управління для перевірки даних	**Опис**
Compare Validator	Порівнює значення, введене в один елемент управління із значенням, введеним в інший елемент, або з фіксованим значенням
CustomValidator	Дозволяє задавати призначений для користувача метод перевірки значень, що вводяться
RangeValidator	Перевіряє, що значення, введене користувачем, знаходиться між двома величинами
RegularExpression Validator	Перевіряє введене значення на відповідність вказаному шаблону
RequiredFieldValidator	Перевіряє обов'язкову наявність введеного значення
ValidationSummary	Відображає звіт про всі помилки перевірки значень, які відбулись на веб-сторінці

Елементи управління веб-сервера

Перелік доступних елементів такого типа приведений в наступній таблиці.

Елемент управління веб-сервера-сервера	**Опис**
AdRotator	Відображення випадково вибраних рекламних оголошень
Button	Відображення кнопки
Calendar	Відображення календаря
CalendarDay	Елемент вибору дня календаря
CheckBox	Відображення прапорця
CheckBoxList	Група прапорців
DataGrid	Відображення полів джерела даних
DataList	Відображення елементів з джерела даних за допомогою шаблонів
DropDownList	Випадний список
HyperLink	Гіперпосилання
Image	Зображення
ImageButton	Кнопка у вигляді зображення
Label	Відображення статичного вмісту, доступного для програмування, і з можливістю завдання стилю
LinkButton	Кнопка з гіперпосилання
ListBox	Випадний список з одиничним або множинним виділенням
ListItem	Елемент списку
Literal	Відображення статичного вмісту, доступного для програмування, але без можливості завдання стилю
Panel	Контейнер для інших елементів управління
PlaceHolder	Місце для додавання елементів управління програмним способом
RadioButton	Радіо-кнопка
RadioButtonList	Група радіо-кнопок
BulletedList	Маркерний список
Repeater	Повторюваний список елементів
Style	Завдання стилю елементу управління
Table	Таблиця
TableCell	Елемент таблиці
TableRow	Рядок таблиці
TextBox	Поле для введення тексту
Xml	Відображення XML файлу або результату XSLT перетворення

3 Зміст роботи

1. Вивчити основи ASP.Net та HTML .

2. Реалізувати прототип веб-інтерфейсу додатка засобами Visual Studio 2008 на ASP.Net (C#) на основі принципів проектування інтерфейсу, описаних в попередній лабораторній роботі. Інтерфейс має бути достатній для виконання всіх сценаріїв з лабораторної роботи № 3. Для зберігання даних можна використовувати СУБД MS SQL Server.

3. Перед реалізацією розглянути декілька варіантів розміщення форм на екрані. Порівняти їх з використанням моделі моделі GOMS. Розрахунок виконати згідно з прикладом наведеним в [1] (стр. 68-80, файл hmi_lections.pdf в директорії book). Розрахунок часу на переміщення курсору з використанням маніпулятора типа «миша» виконати згідно законів Фітса і Хіка (стр. 93-95, файл hmi_lections.pdf в директорії book).

4. Оформити звіт про виконану роботу, що включає приклад використання, кожного з основних принципів проектування інтерфейсу.

5. Захистити лабораторну роботу.

4 Контрольні питання

1. У чому полягають основні відмінності веб-інтерфейсу від інтерфейсу windows-додатку?

2. Якими перевагами володіє веб-інтерфейс порівняно з интерфейсом windows-додатку?

3. Якими недоліками володіє веб-інтерфейс порівняно з интерфейсом windows-додатку?

4. У яких випадках доцільно застосовувати веб-інтерфейс?

5. Які елементи інтерфейсу можуть використовуватися при побудові веб-інтерфейсу?

6. Чи відрізняються ці елементи веб-інтерфейсу від відповідних елементів windows-додатку?

7. Яким чином виконується обробка подій для елементів

веб-інтерфейсу?

8. Яку роль грає HTML в побудові веб-інтерфейсу?

9. Яким чином виконується перевірка даних, що вводяться користувачем у веб-додатку? У чому полягають відмінності даного способу перевірки від перевірки даних в windows-додатку?

Література

1. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.
2. Інтерфейс "Користувач-комп'ютер": Навчальний посібник / В.П. Майданюк, А.М. Пєтух. - Вінниця: ВДТУ, 1999. - 66 с.
3. Мандел Т. Разработка пользовательского интерфейса / Т. Мандел. - Пер. с англ. – М.: ДМК Пресс, 2001. – 416 с.
4. Коутс Р. Интерфейс "Человек - компьютер" / Р. Коутс, И. Влейминк. - Пер. с англ. - М.: Мир, 1990. – 501 с.
5. Буч Грейди, Рамбо Джеймс, Джекобсон Айвар. Язык UML. Руководство пользователя. М.: ДМК, 2000
6. Троелсен Э. C# и платформа .NET. Библиотека программиста. / Пер. с англ. - СПб.: Питер, 2002. - 800 с.

ЛАБОРАТОРНА РОБОТА №5

1 Мета роботи: знайомство з мовними інтерфейсами та Microsoft SAPI.

2 **Методичні вказівки**

Загальні поняття

Звичайне введення даних в комп’ютер за допомогою клавіатури далеко не досконале. Це повільно, досить утомливо і не завжди надійно. Ще в 2000 році група учених Массачусетського університету на семінарі, присвяченому проблемам взаємодії людини і комп'ютера, шукала альтернативу клавіатурі. У той час був запропонований деякий анімований образ людини, з якою користувач міг би вести інтерактивне спілкування за допомогою голосу, а не виробляти звичайний обмін командами.

Втім, учені часто черпали теми для своїх винаходів у фантастів, і ідея створення голосового інтерфейсу, неодмінного атрибуту фантастичних романів і фільмів, виникла ще на ранньому етапі розвитку комп'ютерної техніки, коли не було навіть клавіатури.

Проте сьогодні, коли технічні можливості начебто вже дозволяють реалізувати цю ідею, виявляється, що і такий спосіб управління далекий від ідеалу. Навіть найоптимальніший алгоритм не дозволить уникнути помилок при трактуванні голосових команд, а в деяких випадках управління голосом відбуватиметься навіть повільніше, ніж введення даних звичними способами.

В результаті, ідея створення голосового інтерфейсу сьогодні існує лише у вигляді нечисленних розробок для людей з обмеженими можливостями, тих, які не можуть користуватися звичайною клавіатурою і мишкою.

При цьому важливо розуміти, що реальне використання системи мовного введення потрібне лише для вирішення принципово нових завдань на принципово іншому устаткуванні. Інакше, не дивлячись на теоретичну

важливість проекту, доцільність його практичного вживання буде вельми сумнівна – швидкість введення тексту «з голосу» ненабагато перевищує звичайний машинописний спосіб.

Голосовий інтерфейс Windows 7

У Windows 7 на відміну від XP, де тільки можна було вводити голосом тексти, можна управляти комп'ютером голосом. Можна виголошувати команди, які повинен виконувати комп'ютер, і диктувати текст комп’ютеру.

Перед початком роботи з программою розпізнавання мови Windows необхідно підключити до комп'ютера мікрофон. Після налаштування мікрофону можна навчити комп'ютер краще розуміти голос, створивши для цього мовний профіль, який використовуватиметься комп'ютером для розпізнавання голосу і мовних команд.

Після налаштування мікрофону і створення мовного профілю можна використовувати програму розпізнавання мови для наступних цілей:

- управління комп'ютером. Програма розпізнавання мови сприймає команди і відповідає на них. За допомогою програми розпізнавання мови можна запускати програми і взаємодіяти з Windows.
- диктування і редагування тексту. Можна використовувати програму розпізнавання мови для диктування слів в програми текстових процесорів або для заповнення веб-сервер-форм у веб-серверi-браузері. Також можна використовувати програму розпізнавання мови для редагування тексту на комп'ютері.

Налаштування розпізнавання мови

Перед початком використання програми розпізнавання мови, необхідно налаштувати комп’ютер для програми розпізнавання мови Windows. Три кроки для налаштування програми розпізнавання мови: налаштувати мікрофон, навчитися розмовляти з комп'ютером і навчити комп'ютер розуміти вашу мову.

Налаштування мікрофону

1. Відкрийте параметри розпізнавання мови. Для цього натискуйте кнопку Пуск, виберіть Панель управління, Спеціальні можливості и Розпізнавання мови.
2. Клацніть команду Налаштування мікрофону.
3. Слідуйте інструкціям на екрані.

Якість розпізнавання мови прямо пов'язана з якістю використовуваного мікрофону. Для розпізнавання мови використовуються два основні типи мікрофонів – головні мікрофони і настільні мікрофони. Головні мікрофони найбільш зручні для роботи з програмою розпізнавання мови, оскільки вони найменш схильні до дії зовнішніх звуків.

Навчання розмові з комп'ютером

Windows поставляється із спеціальним підручником, який допомагає освоїти команди, використовувані для розпізнавання мови. Проходження підручника займає приблизно 30 хвилин. Аби запустити підручник по роботі з програмою розпізнавання мови, виконаєте наступні дії:

1. Відкрийте параметри розпізнавання мови. Для цього виберіть кнопку Пуск, Панель управління, Спеціальні можливості і Розпізнавання мови.
2. Клацніть команду Використання мовних підручників.
3. Слідуйте інструкціям підручника по розпізнаванню мови.

Налаштування комп'ютера на розпізнавання мови

Програма розпізнавання мови використовує унікальний мовний профіль для розпізнавання вашого голосу і мовних команд. У міру використання програми розпізнавання мови ваш мовний профіль стає детальнішим, що повинно поліпшити здатність комп’ютера розуміти вас.

1. Відкрийте параметри розпізнавання мови. Для цього виберіть Пуск, Панель управління, Спеціальні можливості і Розпізнавання мови.
2. Клацніть команду «Навчите комп’ютер краще розуміти вас».

3. Слідуйте інструкціям на екрані.

API для підтримки мовного інтерфейсу

Інтерфейс мовного прикладного програмування(Speech Application Programming Interface або SAPI) це API , що розвивається компанією Microsoft для розпізнавання і синтезу мови. На сьогоднішній день існує ряд версій SAPI. Додатки, які використовують SAPI, включають Microsoft Office, Агент Microsoft та інші.

Взагалі всі версії API проектувалися так, що розробник програмного забезпечення може написати додаток розпізнавання і синтезу мови з використання стандартного набору інтерфейсів, доступних з різноманітних мов програмування.

Версії SAPI Microsoft можна розділити на дві групи:

- версії SAPI 1 - SAPI 4 - подібні одна до одної, з додатковими особливостями в кожній новішій версії;
- SAPI 5 - цілком новий інтерфейсом, який з'явився у 2000 році. З того часу вийшло декілька підверсій цього програмного інтерфейсу.

У версії SAPI1 до 4, додатки можуть безпосередньо спілкуватися з двигунами. В SAPI 5 проте, програми та системи не безпосередньо спілкуються один з одним, а з використанням компоненту часу виконання (sapi.dll). API реалізовані цим компонентом саме і використовують програми.

SAPI сумісна з наступними операційними системами:

SAPI 5

- Microsoft Windows 7
- Microsoft Windows Vista
- Microsoft Windows 2003
- Microsoft Windows XP
- Microsoft Windows 2000

SAPI 4

- Microsoft Windows 9x

- Microsoft Windows NT 4.0.

3 Зміст роботи

1. Вивчити основи роботи з мовним інтерфейсом Windows.
2. Налаштувати інтерфейс на виконання різних задач.
3. Ознайомитись з SAPI.
4. Реалізувати найпростіший додаток для розпізнавання і синтезу мови з використанням SAPI у середовищі Visual Studio 2008, мова програмування C++.
5. Оформити звіт про виконану роботу.
6. Захистити лабораторну роботу.

4 Контрольні питання

1. Які перспективи мовного інтерфейсу?
2. Для кого в основному призначений голосовий інтерфейс?
3. Що таке SAPI?
4. Які відмінності між версіями SAPI?
5. Етапи налаштування мовного інтерфейсу Windows?

Література

1. Мандел Т. Разработка пользовательского интерфейса / Т. Мандел. - Пер. с англ. – М.: ДМК Пресс, 2001. – 416 с.
2. Троелсен Э. C# и платформа .NET. Библиотека программиста. / Пер. с англ. - СПб.: Питер, 2002. - 800 с.
3. http://windows.microsoft.com/ru-RU/windows7/Set-up-Speech-Recognition
4. http://windows.microsoft.com/ru-RU/windows7/Use-Speech-Recognition-to-operate-windows-and-programs
5. http://msdn.microsoft.com/en-us/magazine/cc163663.aspx
6. http://msdn.microsoft.com/en-us/library/ms723627(v=vs.85).aspx

7. http://en.wikipedia.org/wiki/Microsoft_Speech_API

ЛАБОРАТОРНА РОБОТА №6

Опис діалогу з використанням UML

Мета роботи: вивчення особливостей опису інтерфейса користувача з використанням діаграм варіантів використання (use case) та діаграм діяльності

Загальні відомості

Уніфікована мова моделювання UML (Unified Modeling Language) – це графічна мова для визначення, подання, проектування і документування програмних систем, організаційно-економічних систем, технічних систем і інших систем різної природи. UML містить стандартний набір діаграм і нотацій найрізноманітніших видів. Текст опису UML кожної версії знаходиться у вільно поширюваних документах, доступних за адресою http://www.omg.org.

UML використовують в 3-х режимах: режим ескізу, режим проектування і режим мови програмування. Безумовно, найголовніший з трьох, – це режим використання UML для ескізування. У цьому режимі розробники використовують UML для обміну інформацією про різні аспекти системи. У режимі проектування можна використовувати ескізи при прямій і зворотній розробці. При прямій розробці (forward engineering) діаграми рисують до написання коди, а при зворотній розробці (reverse engineering) діаграми будуються на підставі вихідного коду аби краще зрозуміти його.

Аби підкреслити, що UML мова графічна, автори називають правила запису (рисування) моделей не синтаксисом, а нотацією. При розробці UML були запропоновані і прийняті розумні рекомендації по вибору нотації. Автори виходили з того, що UML використовуватиметься по різному: починаючи від не дуже акуратного рисування від руки на листку паперу, друк чорно-білих зображень в книгах і закінчуючи створенням складних діаграм за допомогою комп’ютера. Тому як основні графічні елементи були вибрані такі, які було б легко використовувати у всіх випадках. Типів елементів нотації чотири:

- фігури;
- лінії;
- значки;
- тексти.

Склад канонічних діаграм UML такий (рис. 1):

- Діаграма використання
- Діаграма класів
- Діаграма об’єктів
- Діаграма станів
- Діаграма діяльності
- Діаграма послідовності
- Діаграма кооперації
- Діаграма компонентів
- Діаграма розміщення

Рисунок 1 – Класифікація діаграм

На етапі розробки ескізу інтерфейсу користувача представляють інтерес діаграми варіантів використання і діаграми діяльності.

Варіант використання (use case) – це відповідь ПЗ, на подію, що ініціюється ззовні. Варіант використання описує типову взаємодію між користувачем і ПЗ. Він відображає уявлення про поведінку системи з точки зору користувача. На діаграмах варіанти використання представляються у вигляді **овалів**.

Дійова особа – це роль, яку користувач грає по відношенню до системи. На діаграмах варіантів використання вони зображуються у вигляді стилізованих людських фігурок. Дійовою особою може бути користувач людина, зовнішня програмна система або час, якщо запуск яких-небудь подій в системі залежить від часу.

Діаграми варіантів використання показують, які дійові особи ініціюють варіанти використання (від них йде стрілка до варіанту використання). З діаграм зрозуміло, які дійові особи отримують дані в ході виконання варіанту використання (до них йде стрілка від варіанту використання).

Сценарії використання

Сценарій - це один із способів опису структури завдання [5]. Це розповідь про здійснювані дії, це історія, епізод, що відбувається в даних тимчасових рамках і у даному контексті. Різні форми сценаріїв широко застосовуються при розробці програмного забезпечення. Сценарії завдань і взаємодій зазвичай багаті характеристиками і володіють високою реалістичністю.

Сценарії при розробці інтерфейсу користувача описують взаємодію між користувачем (або типом користувачів) і системою. Звичайні сценарії володіють деякими серйозними обмеженнями при спробі використовувати їх для проектування призначеного для користувача інтерфейсу. У них робиться наголос на реалістичність і деталі, при цьому на серйозні проблеми і загальну організацію звертається недостатньо уваги. Сценарії включають правдоподібні описи комбінацій окремих дій і завдань, тому часто буває важко виділити і зрозуміти основну суть взаємодії.

Моделі use case

Концепція моделей use case вперше була застосована для розробки ПЗ Айваром Якобсоном як складова частина його об'єктно-орієнтованого підходу до програмної інженерії. Успіх моделі виявився настільки значним, що з часом сталася інтеграція елементів use case практично у всі основні методи об’єктно-орієнтованого аналізу і проектування. Не дивлячись на те що модель була розроблена спеціально для проектування об'єктно-орієнтованого ПЗ, нічого особливо «об'єктно-орієнтованого» в елементах use case немає, тому їх можна застосовувати практично до всіх підходів до проектування.

Елемент use case - це ситуація, варіант використання, тобто деякий випадок вживання системи. По суті use case це:

- забезпечення функціональності;
- суто зовнішня точка зору (принцип «чорного ящика»);
- оповідний опис;
- опис взаємодії між користувачем (у якійсь ролі) і системою;
- завершене і зрозуміле користувачеві використання системи.

Кожен елемент use case описує в оповідній формі завершену взаємодію, що має ясну мету з точки зору користувача. При об’єктно-орієнтованому підході елементи use case можуть описувати взаємодію з іншими системами і устаткуванням, а не лише з живими користувачами. Проте, коли метою є розробка інтерфейсу користувача, можна абсолютно спокійно обмежитися розглядом лише тих елементів use case, які відносяться до взаємодії людини і системи.

Сутнісні елементи use case

Сутнісний елемент use case - це структурований опис, виражений на мові даної прикладної області і користувачів системи, який містить спрощений, узагальнений, абстрактний, не залежний від технології і реалізації опис однієї завершеної, наповненою сенсом і добре визначеної з точки зору користувачів задачі або взаємодії. Передбачається, що користувач грає певну роль по відношенню до системи, а в описі втілюються цілі і задуми його основної взаємодії.

Сутнісні елементи use case будуються на основі цілей і завдань користувача, а не на основі якихось конкретних механізмів або етапів, що ведуть до досягнення цих цілей.

В сутнісних елементах use case можна виділити три частини: виклад загальних устремлінь користувача та опис, що складається з двох частин і включає модель устремлінь користувача і модель обов'язків системи. Сутнісні елементи use case іменуються, причому за допомогою цих імен прагнуть виразити призначені для користувача наміри в умовах даного варіанту використання. Відповідно до угоди, запропонованої Якобсоном, елемент use case зображається у вигляді еліпса з ім'ям елементу.

Опис варіантів використання

Функціональні вимоги до системи моделюються і документуються за допомогою варіантів використання (use case). Варіант використання (use case) – зв'язний елемент функціональності, що надається системою при взаємодії з дійовими особами. Дійова особа (actor) - роль, узагальнення елементів зовнішнього оточення системи, яке веде себе по відношенню до системи однаковим чином.

У контексті процесу управління вимогами варіанти використання трактуються таким чином (згідно Коберну [2]):

- варіант використання фіксує угоду між учасниками проекту відносно поведінки системи;
- варіант використання описує поведінку системи при різних умовах, коли система відповідає на запит одного з учасників, який називається основною дійовою особою;
- основна дійова особа ініціює взаємодію з системою, щоб досягти деякої мети. Система відповідає, дотримуючись інтересів всіх учасників.

Варіанти використання - це вид документації, який використовується, коли потрібно сконцентрувати зусилля на обговоренні принципових вимог до системи, що розробляється, а не на детальному їх описі. Стиль їх написання залежить від масштабу, кількості учасників і критичності проекту. У загальному випадку, рекомендується дотримуватися наступних правил:

- назви варіантів використання мають бути діловими (нетехнічними ми) термінами, що мають значення для замовника;
- кожен варіант використання має бути завершеною транзакцією між користувачем і системою, що представляє для першого деяку цінність;
- добре написаний варіант використання легко читається і складається з речень, написаних в єдиній граматичній формі. На навчання читанню варіанту використання не повинно витрачатись більше декількох хвилин.

Формат опису варіанту використання такий [2]:

1. Ім'я - мета у вигляді короткої активної дієслівної фрази.
2. Контекст використання - довший опис мети.
3. Зона дії.
4. Рівень точності.

5. Основна дійова особа.
6. Інші учасники і їх інтереси.
7. Передумова (визначає, виконання якої умови гарантує система перед тим, як дозволити запуск варіанту використання).
8. Мінімальні гарантії (найменші обіцянки системи учасникам, зокрема, коли мета основної дійової особи не може бути досягнута).
9. Гарантії успіху (або постумова - postcondition - встановлює, що інтереси учасників задовольняються після успішного завершення варіанту використання в кінці основного сценарію).
10. Тригер (подія, яка запускає варіант використання).
11. Основний сценарій або потік (простий для розуміння типовий сценарій, в якому досягається мета основної дійової особи і задовольняються інтереси всіх учасників). Кожен крок основного сценарію описує: взаємодію двох дійових осіб ("Клієнт вводить адресу"); крок підтвердження для захисту інтересу учасника ("Система підтверджує PIN-код"); внутрішня зміна для задоволення інтересу учасника ("Система виводить суму з балансу").
12. Розширення (запускаються при виникненні певної умови, містять послідовність кроків, які описують, що відбувається за цієї умови, і закінчується досягненням мети або відмовою від неї).
13. Список змін в технології і даних.
14. Допоміжна інформація.

Карти елементів use case

Елементи use case не існують окремо від зовнішнього світу. Повноцінна програмна система повинна забезпечувати підтримку десятків, а то і сотень елементів use case, причому, усередині якимсь чином пов’язаного з нею додатку повинен існувати зв'язок між цими елементами. Відображення взаємозв’язку між додатками дає можливість описати загальну структуру завдання, що вирішується додатком і його інтерфейсом. Карта елементів use case для даного завдання розбиває всі функціональні можливості системи на безліч взаємозв’язаних сутнісних елементів use case. Виділивши всі важливі взаємодії, їх відмінності і показавши стосунки між ними, можна створити спрощену загальну модель завдань, що вирішуються системою, і можливостей, які вона зобов’язана надати.

Повноцінна модель use case є безліччю описів, що визначають суть всіх елементів use case, і карту цих елементів, яка показує стосунки між ними. Між сутнісними елементами use case можуть існувати стосунки різних типів, включаючи спеціалізацію, розширення, композицію, а також схожість. Знання цих стосунків дозволяє аналітику або розробникові виділити загальні елементи завдання і у результаті створити простішу модель завдання.

Спеціалізація

Деякі елементи use case можуть бути спеціалізованими версіями інших елементів. Наприклад, при розробці додатка «банкомат» елементи use case «отриманняГрошей», «розміщенняГрошей» і «запитСтану» є субкласами, або

спеціалізованими варіантами абстрактного класу взаємодій, який може бути названий «використанняБанкомата». Що стосується відношення між елементами «отриманняГрошей» і «використанняБанкомата», то його можна охарактеризувати як класифікацію, або спеціалізацію. Такий тип відношення означає, що один елемент use case «є» («is-a») спеціалізацією іншого. У об'єктно-орієнтованому аналізі і проектуванні таке відношення відповідає відношенню клас-підклас.

Спеціалізація дає можливість спростити загальну модель use case шляхом відділення загальних або універсальних форм взаємодії від специфічних форм, адаптованих для вужчого вживання. Таким чином, немає необхідності переписувати заново найзагальніші патерни застосування. Достатньо написати їх один раз, а потім лише «повторно використовувати» («reuse»), посилаючись на них. Як видно з рис. 2, для відображення відношення спеціалізації використовується подвійна стрілка. Поряд із стрілкою можна зустріти підпис «is-a» або «specialize», залежно від контексту.

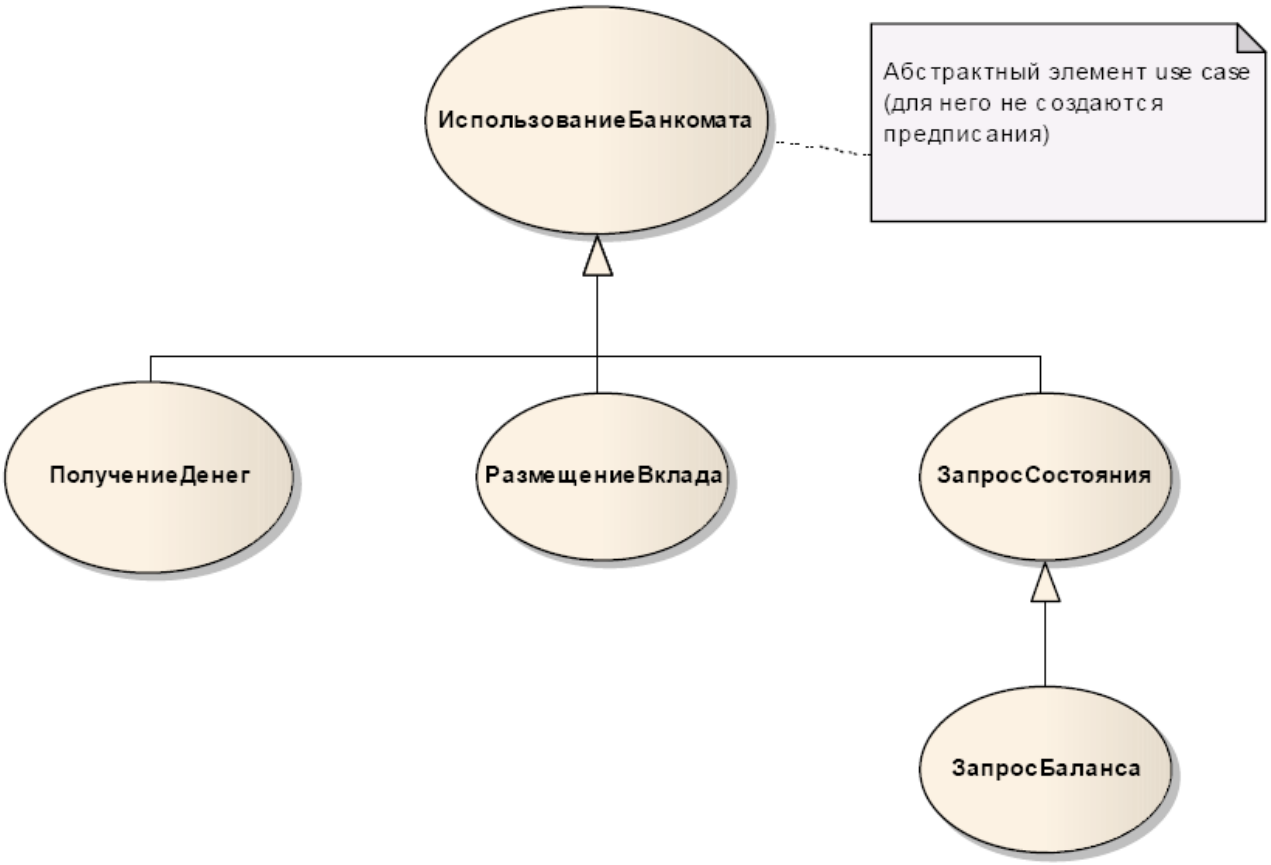

Рисунок 2 – Відношення спеціалізації

Розширення

Однією з інновацій в об'єктно-орієнтованій програмній інженерії, навіяних ідеями Якобсона, стало визнання розширення одним з можливих відношень між елементами use case. Говорять, що один елемент «розширює» інший, коли він містить альтернативні патерни взаємодії, які увійдуть до розширюваного елементу. Наприклад, при реалізації елементу use case для зміни зовнішнього вигляду деякої частини екрана користувачеві необхідно займатися пошуком по всій системі файлу, який містить потрібну картинку або значок. Нормальний, або очікуваний розвиток подій (забезпечується базисом, або базисним елементом use case), але зовсім не обов'язковий.

Розширення - це вдала концепція, що дозволяє значно спростити сутнісні моделі use case. На карті елементів use case відношення расширения зображається у вигляді пунктирної лінії із стрілкою і має підпис «extend», як показано на рис. 3. Якщо для розширення дається додатковий опис, то в нього може бути включена примітка, що показує, які елементи use case розширюються.

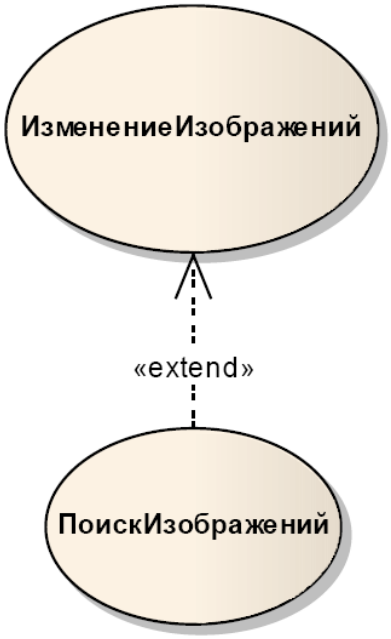

Рисунок 3- Відношення розширення

Композиція

Елементи use case можна декомпозувати на складові частини, або піделементи, що є підлеглими або включеними патернами взаємодії. Відношення композиції позначається на карті елементів use case пунктирною стрілкою, що вказує на піделемент use case і має мітку «include» як показано на рис. 4.

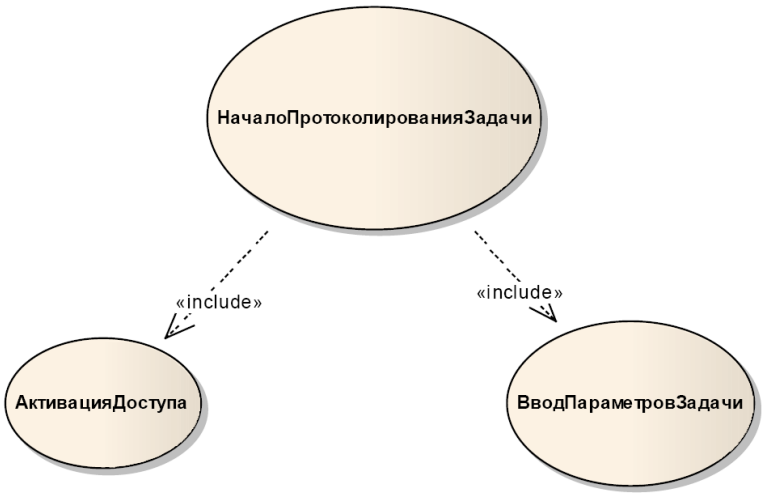

Рисунок 4 – Відношення композиції

Взаємодія, що описується суперелементом use case, здійснюється за допомогою взаємодій, що входять в піделемент або піделементи, причому опис суперелементу посилатиметься на всі піделементи, що використовуються. Наприклад, елемент use case під назвою «початокПротоколюванняЗадачі», створений для програми, що відстежує хід виконання завдань, може використовувати елементи «авторизаціяДоступа» і «введенняПараметрівЗадачі». Такий спосіб моделювання взаємодій дозволяє розділити незалежні і майже ніяк не зв’язані між собою підзадачі «авторизаціяДоступа» і «введенняПараметрівЗадачі».

Діаграми діяльності

Для опису функціональних вимог окрім діаграм варіантів використання застосовуються діаграми діяльності [4]. Зокрема:

- для опису поведінки, що включає велику кількість паралельних процесів;
- для аналізу варіанту використання (описують послідовність дій і їх взаємозв'язок);
- для аналізу потоків робіт (workflow) в різних варіантах використання.

Коли варіанти використання взаємодіють один з одним, діаграми діяльності є засобом подання і аналізу їх поведінки. Приклад діаграми діяльності подано на рис. 5.

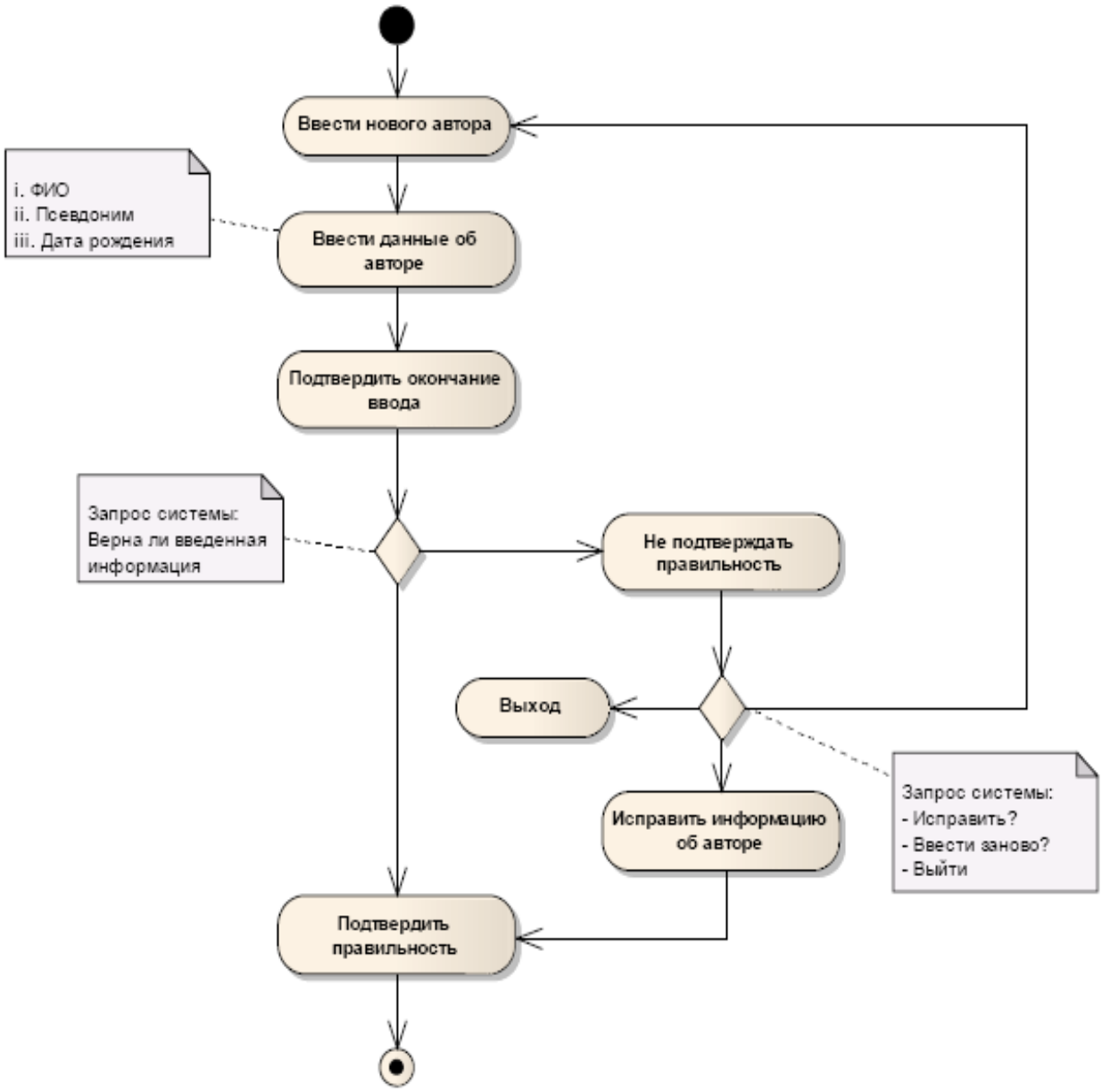

Рисунок 5 – Діаграма діяльності

3 Зміст роботи

1. Вивчити основи побудови use case діаграм і діаграм діяльності.
2. Побудувати use case діаграми.
3. Побудувати діаграми діяльності для кожного варіанту використання з попередньої лабораторної роботи.
4. Оформити звіт про виконану роботу.
5. Захистити лабораторну роботу.

4 Контрольні питання

1. Яким чином виконується моделювання завдань?
2. Що таке сценарій використання?
3. Що таке елемент use case?
4. Що таке сутнісні елементи use case?
5. Чим відрізняються сценарії використання від моделі use case?
6. Яким чином можна описати варіанти використання?
7. Наведіть приклад опису варіанту використання по Коберну?
8. Що таке карта елементів use case?
9. Що означає роль на use case діаграмі?
10. У чому полягає суть відношення спеціалізації? Наведіть приклад.
11. У чому полягає суть відношення розширення? Наведіть приклад.
12. У чому полягає суть відношення композиції? Наведіть приклад.
13. Чим відрізняється відношення спеціалізації від розширення?
14. Що є діаграма діяльності?
15. У чому полягають відмінності use case діаграми від діаграми діяльності?

Література

1. Л. Константайн, Л. Локвуд. Разработка программного обеспечения.: Пер. с англ. - СПб.: Питер, 2004. - 592 с.
2. Коберн А. Современные методы описания функциональных требований к системам.: Пер. с англ. - М. Лори, 2002. - 263 с.
3. Якобсон А., Буч Г., Рамбо Дж. Унифицированный процесс разработки программного обеспечения.: Пер. с англ. - СПб.: Питер, 200 с.- Главы 6, 7.
4. Фаулер М. UML. Основы. 3-е изд. Пер. с англ. - СПб.: Символ-Плюс, 2004. - 192 с.
5. Человеко-машинное взаимодействие: Учебное пособие / Новосибирский государственный технический университет. – Новосибирск, 2006. – 97 с.